

CSS ANIMATIONS

💡 **STUDY HACK:** Read → Copy → Run → Change one value → Observe the difference



LEARN

Understand the property first



TRY

Run the example in browser



REMEMBER

Use the Viva points

1. CSS Animation Basics



Think like the browser: @keyframes tells the browser the journey; animation tells it how to travel.

1. Definition: CSS Animation allows an element to gradually change from one style to another over time.

2. Purpose / Why we use it: To create movement, color changes, fading, rotation, scaling and other visual effects without JavaScript.

3. Beginner-Friendly Explanation: An animation has two main parts: @keyframes defines WHAT changes; animation properties define HOW and WHEN it runs.

4. Syntax:

```
@keyframes name {  
  from { ... }  
  to { ... }  
}  
  
.element {  
  animation: name 3s;  
}
```

5. Example:

```
@keyframes colorChange {  
  from { background-color: red; }  
  to { background-color: yellow; }  
}  
  
.box {  
  width: 100px;  
  height: 100px;  
  animation: colorChange 3s;  
}
```

6. Expected Output: The box changes gradually from red to yellow over 3 seconds.

7. Attributes / Properties Used:

Property / Value	Belongs to	Meaning
@keyframes	CSS at-rule	Defines animation stages.
from	@keyframes stage	Starting state; same as 0%.
to	@keyframes stage	Ending state; same as 100%.

animation-name	CSS property	Selects the keyframes animation.
animation-duration	CSS property	Sets the time for one cycle.

8. Attribute → Belongs to → Meaning: The table above lists the properties and values used in this topic.

9. Important Rules:

- @keyframes needs a name.
- Duration should normally be greater than 0s for a visible animation.
- Percentage stages such as 0%, 50% and 100% can also be used.

10. Common Mistakes:

- Creating @keyframes but not applying it.
- Forgetting the animation duration.
- Using a different animation name in the selector.

11. Viva / Remember Points:

- @keyframes = WHAT changes.
- animation properties = HOW and WHEN it runs.
- 0% = start and 100% = end.

2. Part A — Background Color: Red → Yellow

 **Experiment: change yellow to blue and watch the final color change.**

1. Definition: An animation can change the background-color of an element between two or more colors.

2. Purpose / Why we use it: To demonstrate a simple color-changing animation using keyframes.

3. Beginner-Friendly Explanation: At 0% the box is red and at 100% it is yellow. The browser generates the intermediate frames automatically.

4. Syntax:

```
@keyframes colorChange {
  0% { background-color: red; }
  100% { background-color: yellow; }
}

.box {
  animation: colorChange 3s;
}
```

5. Example:

```
@keyframes colorChange {
  0% { background-color: red; }
  100% { background-color: yellow; }
}

.box {
  width: 150px;
```

```
height: 150px;
animation: colorChange 3s;
}
```

6. Expected Output: The box changes smoothly from red to yellow.

7. Attributes / Properties Used:

Property / Value	Belongs to	Meaning
background-color	CSS property	Sets the background color.
red	CSS color value	Starting color.
yellow	CSS color value	Ending color.
animation-name	CSS property	Selects colorChange.
animation-duration	CSS property	Sets the animation cycle duration.

8. Attribute → Belongs to → Meaning: The table above lists the properties and values used in this topic.

9. Important Rules:

- 0% is the beginning and 100% is the end.
- Intermediate colors are generated automatically when the browser interpolates the values.

10. Common Mistakes:

- Forgetting animation-name or duration.
- Writing 2 instead of 2s when a time value is required.

11. Viva / Remember Points:

- 0% = start.
- 100% = end.
- The browser creates intermediate frames.

3. Part A — Move Left → Right → Bottom → Left → Top

 Visualize the path as four corners of a box: top-left → top-right → bottom-right → bottom-left.

1. Definition: CSS keyframes can change the position of an element through multiple stages.

2. Purpose / Why we use it: To demonstrate movement through four sides of a container.

3. Beginner-Friendly Explanation: Use a positioned element and change its left/top values at 0%, 25%, 50%, 75% and 100%.

4. Syntax:

```
@keyframes moveBox {
  0%   { left: 0; top: 0; }
  25%  { left: 250px; top: 0; }
  50%  { left: 250px; top: 200px; }
  75%  { left: 0; top: 200px; }
  100% { left: 0; top: 0; }
}
```

5. Example:

```

.area {
  position: relative;
  width: 350px;
  height: 280px;
  border: 2px solid black;
}
.box {
  position: absolute;
  width: 60px;
  height: 60px;
  background-color: red;
  animation: moveBox 6s linear infinite;
}

@keyframes moveBox {
  0%   { left: 0; top: 0; }
  25%  { left: 250px; top: 0; }
  50%  { left: 250px; top: 200px; }
  75%  { left: 0; top: 200px; }
  100% { left: 0; top: 0; }
}

```

6. Expected Output: The box moves clockwise around the four sides and returns to its starting position.

7. Attributes / Properties Used:

Property / Value	Belongs to	Meaning
position: relative	CSS property	Makes the parent the positioning reference.
position: absolute	CSS property	Positions the moving box relative to the positioned parent.
left	CSS property	Controls horizontal position.
top	CSS property	Controls vertical position.
animation	CSS shorthand	Applies name, duration, timing and iteration settings.

8. Attribute → Belongs to → Meaning: The table above lists the properties and values used in this topic.

9. Important Rules:

- The parent should normally have position: relative when the child uses absolute positioning.
- The animation coordinates must fit inside the container.
- The first and last keyframes should match when a continuous loop is wanted.

10. Common Mistakes:

- Forgetting position: absolute on the moving element.
- Using coordinates larger than the container.
- Forgetting a duration.

11. Viva / Remember Points:

- left changes horizontal position.
- top changes vertical position.
- 25%, 50%, 75% create the corner stages.

4. Part A — animation-delay: 2s

 **Remember:** DELAY = wait first. DURATION = animation time.

1. **Definition:** animation-delay specifies how long the browser waits before starting an animation.
2. **Purpose / Why we use it:** To create a 2-second pause before the animation starts.
3. **Beginner-Friendly Explanation:** Positive delay values wait before the first animation cycle begins.

4. Syntax:

```
animation-delay: 2s;
```

5. Example:

```
.box {  
  animation-name: colorChange;  
  animation-duration: 3s;  
  animation-delay: 2s;  
}
```

6. **Expected Output:** The animation waits for 2 seconds and then starts.

7. Attributes / Properties Used:

Property / Value	Belongs to	Meaning
animation-delay	CSS property	Sets the waiting time before the animation starts.
2s	Time value	Two-second delay.

8. **Attribute → Belongs to → Meaning:** The table above lists the properties and values used in this topic.

9. Important Rules:

- Time units such as s or ms are required for non-zero time values.
- Delay affects when the animation begins; it does not make the animation itself slower.

10. Common Mistakes:

- Writing 2 instead of 2s.
- Confusing delay with duration.

11. Viva / Remember Points:

- delay = waiting time.
- duration = time taken by one animation cycle.

5. Part A — animation-iteration-count: infinite

 **Challenge:** replace infinite with 3. How many times does the animation run?

1. Definition: animation-iteration-count controls how many times an animation runs.

2. Purpose / Why we use it: To repeat an animation continuously.

3. Beginner-Friendly Explanation: Use a number for a fixed number of repetitions or infinite to keep repeating until the animation is stopped.

4. Syntax:

```
animation-iteration-count: infinite;
```

5. Example:

```
.box {  
  animation: moveBox 5s linear infinite;  
}
```

6. Expected Output: The animation repeats continuously.

7. Attributes / Properties Used:

Property / Value	Belongs to	Meaning
animation-iteration-count	CSS property	Sets the number of animation cycles.
1	Value	Runs once by default.
2, 3, 4 ...	Number value	Runs the specified number of cycles.
infinite	CSS value	Repeats continuously.

8. Attribute → Belongs to → Meaning: The table above lists the properties and values used in this topic.

9. Important Rules:

- The default value is 1.
- infinite is useful for looping effects such as loaders and screensavers.

10. Common Mistakes:

- Writing 'forever' instead of infinite.
- Expecting infinite to mean the animation becomes faster.

11. Viva / Remember Points:

- iteration-count = number of cycles.
- infinite = keep repeating.

6. Part A — animation-direction: reverse

 **Shortcut:** reverse does not rewrite your keyframes; it changes the playback direction.

1. Definition: animation-direction controls the direction in which keyframes are played.

2. Purpose / Why we use it: To play an animation in reverse or alternate directions.

3. Beginner-Friendly Explanation: reverse makes the animation play from the end keyframe toward the start keyframe.

4. Syntax:

```
animation-direction: reverse;
```

5. Example:

```
.box {  
  animation-name: colorChange;  
  animation-duration: 3s;  
  animation-direction: reverse;  
}
```

6. Expected Output: The animation starts by playing from the final keyframe toward the first keyframe.

7. Attributes / Properties Used:

Property / Value	Belongs to	Meaning
animation-direction	CSS property	Controls the playback direction.
normal	Value	Plays normally from 0% to 100%.
reverse	Value	Plays from 100% to 0%.
alternate	Value	Forward, then backward.
alternate-reverse	Value	Backward, then forward.

8. Attribute → Belongs to → Meaning: The table above lists the properties and values used in this topic.

9. Important Rules:

- reverse does not change the keyframe definitions; it changes how they are played.
- Direction is especially noticeable when more than one keyframe stage is used.

10. Common Mistakes:

- Confusing reverse with right-to-left movement.
- Changing left/top values when only playback direction was required.

11. Viva / Remember Points:

- normal = forward.
- reverse = backward.
- alternate = forward then backward.

7. Part A — animation-timing-function



Think of this as the accelerator/brake of your animation: it controls the speed pattern.

1. Definition: animation-timing-function controls the speed pattern of an animation during a cycle.

2. Purpose / Why we use it: To make an animation move at a constant speed or change speed during its movement.

3. Beginner-Friendly Explanation: linear keeps the speed constant. ease, ease-in, ease-out and ease-in-out create different acceleration and deceleration patterns.

4. Syntax:

```
animation-timing-function: value;
```

5. Example:

```
.box {  
  animation-name: moveBox;  
  animation-duration: 5s;  
  animation-timing-function: ease-in-out;  
}
```

6. Expected Output: The box changes speed according to the selected timing function.

7. Attributes / Properties Used:

Property / Value	Belongs to	Meaning
linear	Timing value	Constant speed.
ease	Timing value	Starts and ends slower, with faster movement in between.
ease-in	Timing value	Starts slowly and speeds up.
ease-out	Timing value	Starts faster and slows down.
ease-in-out	Timing value	Starts slowly, speeds up, then slows down.
step-start	Timing value	Jumps immediately to the next step.
step-end	Timing value	Holds the current step until the end of the step.
steps(n)	Timing function	Divides the animation into a specified number of steps.
cubic-bezier(...)	Timing function	Customizes the speed curve.

8. Attribute → Belongs to → Meaning: The table above lists the properties and values used in this topic.

9. Important Rules:

- linear is easiest for predictable constant movement.
- Use steps() for jump-like or frame-by-frame effects.
- Timing function changes speed over time, not the total duration.

10. Common Mistakes:

- Thinking ease-in means the whole animation is slow.
- Confusing timing function with animation-duration.
- Using steps() without understanding that motion becomes discrete.

11. Viva / Remember Points:

- linear = constant speed.
- ease-in = slow start.
- ease-out = slow end.
- ease-in-out = slow start and slow end.

8. Important Animation Properties — Complete Reference



Animation control panel: Name → Duration → Delay → Count → Direction → Timing → Fill → Play.

1. Definition: CSS provides several animation properties that control the name, time, repetition, direction, timing, finishing state and play state of an animation.

2. Purpose / Why we use it: To understand the complete animation control system instead of memorizing only one shorthand example.

3. Beginner-Friendly Explanation: Think of animation properties as controls: WHAT runs, HOW LONG it runs, WHEN it starts, HOW MANY times it runs, WHICH direction it uses, HOW its speed changes, and WHAT happens before/after it runs.

4. Syntax:

```
animation-name: name;
animation-duration: 3s;
animation-delay: 1s;
animation-iteration-count: infinite;
animation-direction: alternate;
animation-timing-function: ease-in-out;
animation-fill-mode: forwards;
animation-play-state: running;
```

5. Example:

```
.box {
  animation-name: moveBox;
  animation-duration: 5s;
  animation-delay: 2s;
  animation-iteration-count: infinite;
  animation-direction: alternate;
  animation-timing-function: ease-in-out;
  animation-fill-mode: both;
  animation-play-state: running;
}
```

6. Expected Output: The selected animation runs using the complete set of controls.

7. Attributes / Properties Used:

Property / Value	Belongs to	Meaning
animation-name	CSS property	Names the @keyframes animation to use.
animation-duration	CSS property	Sets the time for one cycle.
animation-delay	CSS property	Sets the waiting time before the animation starts.
animation-iteration-count	CSS property	Sets how many cycles run.
animation-direction	CSS property	Controls playback direction.
animation-timing-function	CSS property	Controls speed pattern.
animation-fill-mode	CSS property	Controls styles before/after the animation.
animation-play-state	CSS property	Pauses or runs the animation.

animation	CSS shorthand	Combines animation settings in one declaration.
-----------	---------------	---

8. Attribute → Belongs to → Meaning: The table above lists the properties and values used in this topic.

9. Important Rules:

- Every animation needs a keyframe name and a duration for a normal visible effect.
- Use the longhand properties while learning; use shorthand after understanding them.

10. Common Mistakes:

- Using a property name incorrectly.
- Forgetting the @keyframes name must match animation-name.
- Mixing up delay and duration.

11. Viva / Remember Points:

- name = what animation?
- duration = how long?
- delay = when start?
- iteration-count = how many times?
- direction = which way?
- timing-function = how does speed change?

9. animation-fill-mode

✿ **Think outside the animation: fill-mode controls what style is kept before/after the active animation.**

1. Definition: animation-fill-mode controls whether the element keeps styles from the animation before it starts, after it ends, or both.

2. Purpose / Why we use it: To control the appearance of an element before and after an animation.

3. Beginner-Friendly Explanation: backwards applies the first relevant keyframe during the delay. forwards keeps the final keyframe after the animation finishes. both combines both behaviors.

4. Syntax:

```
animation-fill-mode: value;
```

5. Example:

```
.box {
  animation-name: colorChange;
  animation-duration: 3s;
  animation-delay: 2s;
  animation-fill-mode: both;
}
```

6. Expected Output: The element follows the selected fill behavior during the delay and after the animation.

7. Attributes / Properties Used:

Property / Value	Belongs to	Meaning
none	Value	Does not apply animation styles outside the active period.
forwards	Value	Keeps the final keyframe after the animation ends.
backwards	Value	Applies the first relevant keyframe during delay.
both	Value	Applies both backwards and forwards behavior.

8. Attribute → Belongs to → Meaning: The table above lists the properties and values used in this topic.

9. Important Rules:

- fill-mode is about styles outside the active animation period.
- forwards is useful when the final state should remain visible.

10. Common Mistakes:

- Thinking forwards means forward direction.
- Confusing fill-mode with animation-direction.

11. Viva / Remember Points:

- fill-mode = what style remains before/after.

10. animation-play-state

Mini experiment: use paused, then change it to running and observe the difference.

1. Definition: animation-play-state controls whether an animation is running or paused.

2. Purpose / Why we use it: To pause and resume an animation using CSS.

3. Beginner-Friendly Explanation: running plays the animation. paused stops the animation at its current point instead of resetting it.

4. Syntax:

```
animation-play-state: running;
animation-play-state: paused;
```

5. Example:

```
.box {
  animation: moveBox 5s linear infinite;
  animation-play-state: paused;
}
```

6. Expected Output: The animation remains paused until its play state becomes running.

7. Attributes / Properties Used:

Property / Value	Belongs to	Meaning
------------------	------------	---------

animation-play-state	CSS property	Controls whether the animation is running or paused.
running	Value	Animation plays.
paused	Value	Animation is paused.

8. Attribute → Belongs to → Meaning: The table above lists the properties and values used in this topic.

9. Important Rules:

- paused does not normally restart the animation from the beginning; it freezes the current animation state.
- The property can be changed with a class, pseudo-class or JavaScript.

10. Common Mistakes:

- Confusing paused with display:none.
- Expecting pause to reset the animation.

11. Viva / Remember Points:

- running = play.
- paused = stop temporarily at the current point.

11. Animation Shorthand

 **Pro tip: learn longhand first; shorthand becomes easy once every value has a meaning.**

1. Definition: The animation shorthand property combines several animation properties in one declaration.

2. Purpose / Why we use it: To write animation CSS more quickly and keep related settings together.

3. Beginner-Friendly Explanation: Instead of writing many animation properties separately, you can combine the main values into animation.

4. Syntax:

```
animation: name duration timing-function delay iteration-count direction fill-mode play-state;
```

5. Example:

```
.box {
  animation: moveBox 5s ease-in-out 2s infinite alternate;
}
```

6. Expected Output: The animation runs with the selected name, duration, timing, delay, repetition and direction.

7. Attributes / Properties Used:

Property / Value	Belongs to	Meaning
animation	CSS shorthand	Combines animation-related properties.
moveBox	Animation name	Selects the @keyframes rule.
5s	Duration	One animation cycle takes 5 seconds.
ease-in-out	Timing function	Controls speed pattern.

2s	Delay	Waits 2 seconds before starting.
infinite	Iteration count	Repeats continuously.
alternate	Direction	Plays forward then backward.

8. Attribute → Belongs to → Meaning: The table above lists the properties and values used in this topic.

9. Important Rules:

- Learn the individual properties first, then use shorthand.
- Not every optional value must be written, but the duration should be clearly provided for predictable results.

10. Common Mistakes:

- Putting values in confusing combinations.
- Forgetting the animation name.
- Forgetting the time unit.

11. Viva / Remember Points:

- animation shorthand = many animation properties in one line.

12. Part B — Create a Custom Animation

 **Your turn: change `translateY()` to `translateX()` and make the box move sideways.**

1. Definition: A custom animation is an animation designed by the developer using named @keyframes stages.

2. Purpose / Why we use it: To create an original animation rather than only changing one property.

3. Beginner-Friendly Explanation: Choose the properties you want to animate, divide the movement into stages, name the animation, and apply it to an element.

4. Syntax:

```
@keyframes customName {
  0% { ... }
  50% { ... }
  100% { ... }
}
.element { animation: customName 3s; }
```

5. Example:

```
@keyframes bounceBox {
  0% { transform: translateY(0); }
  50% { transform: translateY(150px); }
  100% { transform: translateY(0); }
}
.box {
  width: 70px;
  height: 70px;
  background-color: orange;
  animation: bounceBox 2s ease-in-out infinite;
}
```

6. Expected Output: The box repeatedly moves down and back up, creating a simple bouncing effect.

7. Attributes / Properties Used:

Property / Value	Belongs to	Meaning
@keyframes bounceBox	CSS at-rule	Defines the custom animation stages.
transform	CSS property	Changes the visual transformation.
translateY()	Transform function	Moves the element vertically.
ease-in-out	Timing value	Makes the movement smoother.
infinite	Iteration value	Repeats the custom animation.

8. Attribute → Belongs to → Meaning: The table above lists the properties and values used in this topic.

9. Important Rules:

- Use meaningful animation names.
- Use percentages when more than two stages are needed.
- Keep the animation duration and movement suitable for the size of the element.

10. Common Mistakes:

- Misspelling the animation name.
- Forgetting to apply the animation to the element.
- Using an extremely large movement that leaves the visible area.

11. Viva / Remember Points:

- Custom animation = your own @keyframes.
- Use 0%, 50%, 100% for simple multi-stage effects.

13. Part C — Screensaver-Style Animation

 **Screensaver recipe: positioned container + moving object + keyframes + infinite loop.**

1. Definition: A screensaver-style animation continuously moves an object around a visible area using repeated keyframes.

2. Purpose / Why we use it: To create a continuous moving visual similar to a simple screensaver.

3. Beginner-Friendly Explanation: Create a large container, place a small object inside it, animate its position, and use infinite repetition so it keeps moving.

4. Syntax:

```
@keyframes screenMove {  
  0% { left: 0; top: 0; }  
  25% { left: 300px; top: 0; }  
  50% { left: 300px; top: 220px; }  
  75% { left: 0; top: 220px; }  
  100% { left: 0; top: 0; }  
}
```

5. Example:

```
body { margin: 0; }
.screen {
  position: relative;
  width: 500px;
  height: 300px;
  border: 3px solid black;
  overflow: hidden;
}
.ball {
  position: absolute;
  width: 50px;
  height: 50px;
  border-radius: 50%;
  background-color: red;
  animation: screenMove 6s linear infinite;
}
@keyframes screenMove {
  0% { left: 0; top: 0; }
  25% { left: 430px; top: 0; }
  50% { left: 430px; top: 250px; }
  75% { left: 0; top: 250px; }
  100% { left: 0; top: 0; }
}
```

6. Expected Output: A small circle continuously travels around the inside boundary of the container.

7. Attributes / Properties Used:

Property / Value	Belongs to	Meaning
position: relative	CSS property	Creates the positioning reference for the moving object.
position: absolute	CSS property	Allows the object to be placed using top/left.
overflow: hidden	CSS property	Keeps content from visibly leaving the container.
border-radius: 50%	CSS property	Makes a square element appear circular.
animation-iteration-count: infinite	CSS property	Keeps the animation repeating.
linear	Timing value	Keeps the movement speed constant.

8. Attribute → Belongs to → Meaning: The table above lists the properties and values used in this topic.

9. Important Rules:

- The container should be large enough for the moving object.
- Coordinate values must account for the object's width and height.
- infinite is important for a continuous screensaver effect.

10. Common Mistakes:

- Using coordinates that push the object outside the visible area.
- Forgetting position: relative on the container.
- Forgetting infinite repetition.

11. Viva / Remember Points:

- Screensaver effect = continuous movement + infinite loop.
- linear gives predictable movement.

14. Complete Part A Practical Example

Purpose: This single example demonstrates background-color change, four-side movement, delay, infinite repetition, reverse direction and timing function.

```
<!DOCTYPE html>
<html>
<head>
<style>
.area {
  position: relative;
  width: 400px;
  height: 300px;
  border: 3px solid black;
}

.box {
  position: absolute;
  width: 70px;
  height: 70px;
  background-color: red;

  animation-name: demo;
  animation-duration: 8s;
  animation-delay: 2s;
  animation-iteration-count: infinite;
  animation-direction: normal;
  animation-timing-function: linear;
}

@keyframes demo {
  0% {
    left: 0;
    top: 0;
    background-color: red;
  }

  25% {
    left: 300px;
    top: 0;
    background-color: yellow;
  }

  50% {
    left: 300px;
    top: 200px;
    background-color: blue;
  }

  75% {
    left: 0;
    top: 200px;
```

```

        background-color: green;
    }

    100% {
        left: 0;
        top: 0;
        background-color: red;
    }
}
</style>
</head>

<body>
    <div class="area">
        <div class="box"></div>
    </div>
</body>
</html>

```

Expected Output: After a 2-second delay, the box continuously travels around the four sides of the container while its background color changes at the keyframe stages.

15. Quick Revision Summary

Concept	Remember
@keyframes	Defines WHAT changes.
animation-name	Selects the keyframes animation.
animation-duration	How long one cycle takes.
animation-delay	How long to wait before starting.
animation-iteration-count	How many times it repeats.
animation-direction	Which direction the keyframes play.
animation-timing-function	How the speed changes.
animation-fill-mode	What styles apply before/after.
animation-play-state	Running or paused.
animation	Shorthand for animation settings.
0%	Starting stage.
50%	Middle stage.
100%	Ending stage.

16. Viva / Remember Points

- @keyframes defines the stages of a CSS animation.
- animation-duration defines the time for one animation cycle.
- animation-delay adds waiting time before the animation begins.
- infinite repeats the animation continuously.
- reverse plays the keyframes in the opposite direction.
- linear keeps animation speed constant.
- ease-in starts slowly; ease-out ends slowly.
- animation-fill-mode controls styles outside the active animation period.
- animation-play-state can be running or paused.
- Use percentage keyframes such as 0%, 25%, 50%, 75% and 100% for multi-stage movement.
- Custom animation = named @keyframes + animation properties.
- Screensaver-style animation commonly uses position changes + infinite repetition.

17. Common CSS Animation Mistakes — Quick Checklist

- Forgetting to write @keyframes.
- Using a different name in animation-name and @keyframes.
- Forgetting animation-duration.
- Forgetting time units such as s or ms.
- Using coordinates without correctly positioning the element.
- Using delay when you actually need a longer duration.
- Confusing reverse direction with reverse physical movement.
- Forgetting infinite when a continuous loop is required.
- Using very large top/left values that move the element outside the container.
- Forgetting to test the animation in the browser.